

LDPC Encoder

Theoretical background

Low-density parity-check (LDPC) codes, known as Gallager codes, are a class of error-correction codes first proposed in 1960 in Robert G. Gallager in his doctoral dissertation at the Massachusetts Institute of Technology. Largely ignored at the time, since iterative decoding algorithm, albeit linear in complexity, were too computationally expensive for the available hardware at the time. They came back in mid 1990s, when hardware improvements were enough that viable implementations were available and they started being used in telecommunication and in other fields, such as DVB-S2 standard for digital television.

LDPC is defined by a sparse parity-check matrix. For linear code $\mathbf{C}$ of length n and dimension k , this matrix $\mathbf{H}$ is an $(n - k) \times n$ matrix. It satisfies the property that a codeword $\mathbf{c}$ in $\mathbf{C}$ is orthogonal to every row of $\mathbf{H}$, meaning:

$$\mathbf{H}\mathbf{c}^T = \mathbf{0}$$

where $\mathbf{0}$ is a zero vector.

Here is an example $\mathbf{H}$ (parity check) matrix for regular LDPC code ($n = 7$, $k = 4$):

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

This matrix can encode certain codewords. To find those we need to find a **null-space** matrix called **generator matrix $\mathbf{G}$** of matrix $\mathbf{H}$. In this case the generator matrix will be this:

$$\mathbf{G} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

Having it, we can try to encode a message $\mathbf{u}$ (of length k) into a codeword $\mathbf{c}$ (of length $m = \text{rows of } \mathbf{G}$) by just multiplying message with generator matrix:

$$\mathbf{c} = \mathbf{u} \cdot \mathbf{G}$$

Ex. $\mathbf{u} = [1, 0, 1, 0]$ gives us codeword $\mathbf{c} = [1, 0, 1, 0, 0, 1, 1]$.

To check if codeword is correct, we can use this equation:

$$s = c \cdot H^T$$

S stands for **codeword syndrome** and if this vector is **zero**, that means codeword is correct.

Goal

The goal of this task is to create full set of correct codewords being provided a parity-check matrix.

Below is an example of solution to some given parity-check matrix **He** (e stands for example):

$$H_e = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

The generator matrix (**Ge**) for this parity-check matrix therefore is:

$$G_e = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Using this matrix, we can generate codewords for all 16 of 4-bit words to encode:

$$\text{Encoded_Words_Matrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 0 & 1 \end{bmatrix}$$

Provided module

You can edit only the *task_15.sv* file. If you wish, you may add other files for supportive submodules.

The module has the following inputs and outputs:

Table 1 List of inputs and outputs of the *task_15* module.

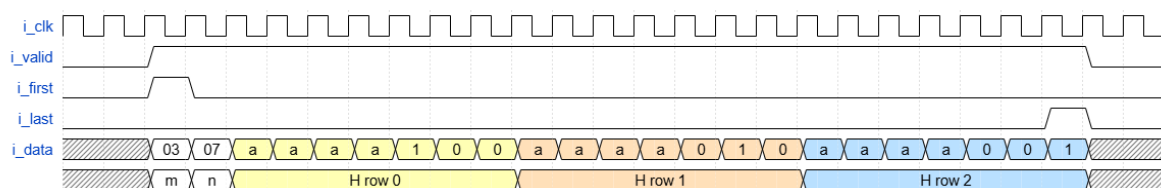
Signal name	Signal type	Bit length
i_clk	Input	1 bit
i_valid	Input	1 bit
i_rst	Input	1 bit
i_data	Input	8 bits
i_last	Input	1 bit
i_first	Input	1 bit
o_data	Output	8 bits
o_valid	Output	1 bit
o_last	Output	1 bit

The reset is synchronous and active high.

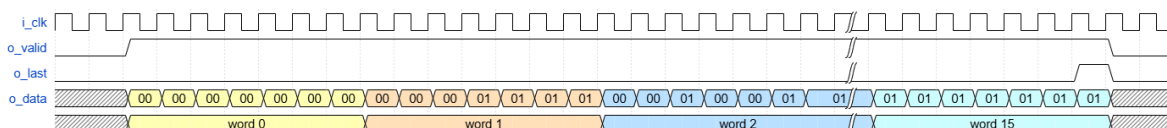
Input and output interfaces

In one packet task receives **from 14 to 198 samples** of input data, 8 bits each. The last sample in a packet is indicated by signal *i_last*. The task should send 136 to 3592 of output samples, 8 bits each, and should indicate the last sample in the output packet by providing signal *o_last*.

Task receives data in the way shown in the figure below, as an example:



To send data from the task in a proper way one needs to provide the *o_valid* signal that is synchronized with data on the *o_data* bus and *o_last* signal that is synchronized with the last sample in the output packet. Below is an output example:



Output data can be sent as a continuous stream, or it can have regular or irregular gaps. The only requirement is providing *o_valid* and *o_last* signals.

Evaluating the task

The judge generates a parity-check matrix, sends it to your design, and independently computes the complete set of encoded words for it. Your answer is compared with that reference byte by byte over the entire $2^k \times n$ output: the score is awarded only if every byte matches, and a single incorrect, missing or extra byte in the stream results in zero points for the test vector. There is no partial credit, so an otherwise correct encoder that emits the codewords in the wrong order, packs bits into bytes, or terminates the stream on the wrong sample will be scored the same as one that produces nothing.

In Development Mode the evaluation is based on whole response being correct and is awarded 5 points.

In Evaluation Mode there are 3 cases that need to be covered, each 5 points.

Additional points may be awarded for lowest resource utilisation when latency does not exceed $21\mu\text{s}$. For more information, check chapter 2.3.2 Bonus Points in the FPGA_Hackathon_2026_Project_Guide document.